

Implementation of e1000 Driver and UDP Networking in xv6

Operating Systems Lab Report

Kanithi Sriarthik

Roll No: CS23B029

Department of Computer Science and Engineering

October 5, 2025

Code Changes I Made

e1000.c

e1000_transmit : the code for this function is completed

```
if(len>DATA_MAX){
    return -1;
}

acquire(&e1000_lock);

int index = regs[E1000_TDT];

if(!(tx_ring[index].status & E1000_TXD_STAT_DD)){
    release(&e1000_lock);
    return -1;
}
if (tx_bufs[index]) {
    kfree(tx_bufs[index]);
    tx_bufs[index] = 0;
}
tx_bufs[index]= (char*)kalloc();
if(tx_bufs[index]==0){
    release(&e1000_lock);
    return -1;
}

memmove(tx_bufs[index],buf,len);

tx_ring[index].addr = (uint64)tx_bufs[index];
tx_ring[index].status =0;
tx_ring[index].length= (uint16)len;
tx_ring[index].cso = 0;
tx_ring[index].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
tx_ring[index].css = 0;
tx_ring[index].special = 0;
regs[E1000_TDT]= (index+1)%TX_RING_SIZE;
release(&e1000_lock);

return 0;
```

e1000_recv : the code for this function is completed

```
acquire(&e1000_lock);

int index = (regs[E1000_RDT]+1)%RX_RING_SIZE;
```

```

while(rx_ring[index].status&E1000_RXD_STAT_DD){
    char* newaddress = (char*)kalloc();
    if(!newaddress){
        break;
    }
    char* address_to_be_passed = rx_bufs[index];
    uint16 len_to_be_passed = rx_ring[index].length;

    rx_bufs[index]=newaddress;
    rx_ring[index].status =0;
    rx_ring[index].addr = (uint64)newaddress;

    regs[E1000_RDT]= index;

    release(&e1000_lock);

    net_rx(address_to_be_passed ,len_to_be_passed);

    acquire(&e1000_lock);

    index = (index+1)%RX_RING_SIZE;
}
release(&e1000_lock);

```

e1000_intr : freeing the tx buffer array after e1000_recv

```

acquire(&e1000_lock);
for(int i = 0; i < TX_RING_SIZE; i++){
    if(tx_bufs[i] && (tx_ring[i].status & E1000_TXD_STAT_DD)){
        kfree(tx_bufs[i]);
        tx_bufs[i] = 0;
    }
}
release(&e1000_lock);

```

net.c

Added the structs required at the start of the file

```

struct udp_packet{
    struct udp_packet* next;
    uint32 src;
    uint16 srcport;
    int len;
    char* data;
};

```

```

struct bind_entry{
    int port;
    struct udp_packet *head;
    struct udp_packet *tail;
    int qlen;
    struct spinlock lock;
    struct bind_entry *nxt;
};

```

sys_bind : completed this function code

```

int portnum;
argint(0,&portnum);
if(portnum<0){
    return -1;
}
acquire(&netlock);
struct bind_entry *itr;
for(itr=bind_list;itr!=0;itr=itr->nxt){
    if(itr->port==portnum){
        release(&netlock);
        return -1;
    }
}
itr=(struct bind_entry*)kalloc();
if(!itr){
    release(&netlock);
    return -1;
}
memset(itr, 0, sizeof(*itr));
itr->port = portnum;
itr->head=itr->tail=0;
initlock(&itr->lock,"bind_entry");
itr->nxt = bind_list;
bind_list = itr;
release(&netlock);
return 0;

```

sys_unbind : completed this function code

```

int port;
argint(0,&port);
acquire(&netlock);

struct bind_entry **prevptr= &bind_list;
struct bind_entry *ptr = bind_list;

```

```

while(ptr){
    if(ptr->port==port){
        *prevptr=ptr->nxt;
        acquire(&ptr->lock);
        struct udp_packet *presentpac=ptr->head;
        while(presentpac){
            struct udp_packet *nextpac=presentpac->next;
            if(presentpac->data) kfree(presentpac->data);
            kfree((char*)presentpac);
            presentpac=nextpac;
        }
        ptr->head=ptr->tail=0;
        release(&ptr->lock);
        kfree((char*)ptr);
        release(&netlock);
        return 0;
    }
    prevptr = &ptr->nxt;
    ptr=ptr->nxt;
}
release(&netlock);
return -1;

```

sys_recv : completed this function code

```

int dstport;
uint64 srcaddr;
uint64 srcport;
uint64 buffaddr;
int maxlen;

argint(0,&dstport);
argaddr(1,&srcaddr);
argaddr(2,&srcport);
argaddr(3,&buffaddr);
argint(4,&maxlen);

struct proc* p = myproc();
acquire(&netlock);

struct bind_entry *itr = bind_list;

while(itr&&itr->port!=dstport){
    itr = itr->nxt;
}

if(!itr){
    release(&netlock);
}

```

```

    return -1;
}

acquire(&itr->lock);
release(&netlock);

while(itr->head==0){
    sleep(itr,&itr->lock);
}

struct udp_packet *presentpacket = itr->head;
itr->head = presentpacket->next;
if(itr->head ==0) itr->tail=0;
itr->qlen--;

release(&itr->lock);

if(copyout(p->pagetable,srcaddr,(char*)&presentpacket->src,sizeof(
    ↪ presentpacket->src))<0){
    goto exit_recv;
}

uint16 srcport_host = presentpacket->srcport;
if(copyout(p->pagetable,srcport,(char*)&srcport_host,sizeof(
    ↪ srcport_host))<0){
    goto exit_recv;
}

int copylen = presentpacket->len;
if(presentpacket->len>=maxlen){
    copylen = maxlen;
}
if(copylen<0) copylen=0;

if(copylen>0){
    if(copyout(p->pagetable,buffaddr,presentpacket->data,copylen)<0){
        goto exit_recv;
    }
}

if(presentpacket->data){
    kfree(presentpacket->data);
}
kfree((char*)presentpacket);

return copylen;

exit_recv:

```

```

if(presentpacket->data){
    kfree(presentpacket->data);
}
kfree((char*)presentpacket);
return -1;

```

sys_send : freed the buffer buf at the end of the function code

```

kfree(buf);

```

ip_rx : completed the function code

```

if(len<(int)((sizeof(struct eth))+sizeof(struct ip)+sizeof(struct udp))
    ↪ ){
    kfree(buf);
    return ;
}

struct eth *eth_header = (struct eth*)buf;
struct ip *ip_header = (struct ip*)(eth_header+1);

if((ip_header->ip_vhl>>4)!=4){
    kfree(buf);
    return;
}

if(ip_header->ip_p==IPPROTO_UDP){

    if(ntohl(ip_header->ip_dst)!=local_ip){
        kfree(buf);
        return;
    }

    int ihl = (ip_header->ip_vhl & 0x0f) <<2;
    if(ihl<(int)sizeof(struct ip)){
        kfree(buf);
        return;
    }

    struct udp *udp_header = (struct udp*)((char*)ip_header+ihl);
    int udplen = ntohs(udp_header->ulen) - sizeof(struct udp);
    if(udplen<0){
        kfree(buf);
        return;
    }

    int dport = ntohs(udp_header->dport);

```

```

int sport = ntohs(udp_header->sport);

acquire(&netlock);
struct bind_entry *itr = bind_list;
while(itr && itr->port != dport){
    itr = itr->nxt;
}

if(!itr){
    printf("bp is 0!\n");
    release(&netlock);
    kfree(buf);
    return;
}

acquire(&itr->lock);
release(&netlock);

struct udp_packet *presentpacket = (struct udp_packet*)kalloc();

if(!presentpacket){
    release(&itr->lock);
    kfree(buf);
    return;
}

presentpacket->next = 0;
presentpacket->src = ntohl(ip_header->ip_src);
presentpacket->srcport = (uint16) sport;
presentpacket->len = udplen;

presentpacket->data = (char*)kalloc();

if(!presentpacket->data || udplen > PGSIZE){
    kfree((char*)presentpacket);
    release(&itr->lock);
    kfree(buf);
    return;
}

if(itr->qlen >= MAX_UDP_QUEUE){
    release(&itr->lock);
    if(presentpacket->data)
        kfree(presentpacket->data);
    kfree((char*)presentpacket);
    kfree(buf);
    return;
}

```



```

char* payload_itr = (char*)(udp_header+1);
memmove(presentpacket->data,payload_itr,udplen);

if(itr->tail){
    itr->tail->next = presentpacket;
}else{
    itr->head = presentpacket;
}
itr->tail = presentpacket;
itr->qlen++;

release(&itr->lock);
wakeup(itr);
}

kfree(buf);

```

net.h

```
#define MAX_UDP_QUEUE 16
```

Code Explanation

e1000.c

- This is basically the code that controls the emulated QEMU NIC. This tells the OS what to do when it wants to send or receive the data.
- **e1000_transmit:** Checks data length, manages descriptor reuse, allocates new pages, updates E1000 registers, and signals NIC to transmit.
- **e1000_recv:** Traverses receive ring descriptors, allocates new buffers, and passes packet data to the network stack.
- **e1000_intr:** Handles interrupts from NIC, frees transmitted buffers when the done bit is set to prevent memory leaks.

net.c

- This is xv6's networking stack where protocol implementations lie.
- **sys_bind:** Maintains a linked list of bound ports, adds new bindings if not already present.

- **sys_unbind:** Removes a binding, frees memory, and updates the list.
- **sys_recv:** Moves UDP packets from kernel queues to user space, copying packet data, src IP, and port.
- **ip_rx:** Called when a packet arrives from NIC. Validates headers, finds the bound port queue, enqueues the packet, wakes up waiting processes, and frees the buffer.